

LlamaIndex Integration

Compress RAG context and chat memory inside LlamaIndex.TS with drop-in components. Smaller prompts, lower LLM spend, faster responses — no changes to your retrievers or vector store.

1 Install

```
npm install @compresr/sdk llamaindex
export COMPRESR_API_KEY=cmp...
```

`llamaindex` is declared as an optional peer dependency (`>=0.8.0`) and is only required if you use this integration. Node.js 20+.

```
import {
  CompresrNodePostprocessor,
  CompresrMemoryBlock,
  wrapToolWithCompresr,
} from '@compresr/sdk/integrations/llamaindex';
```

2 Drop-in node postprocessor

Add `CompresrNodePostprocessor` to `asQueryEngine()` and every retrieved node is rewritten with query-aware compression before the response synthesizer sees it. The user query is read automatically from the query engine's `QueryBundle`.

```
import { VectorStoreIndex, Document } from 'llamaindex';
import { CompresrNodePostprocessor }
  from '@compresr/sdk/integrations/llamaindex';

const index = await VectorStoreIndex.fromDocuments(
  [new Document({ text: longText })],
);

const compresrPp = new CompresrNodePostprocessor({
  apiKey: process.env.COMPRESR_API_KEY!,
  compressionModel: 'latte_v1', // query-aware (default)
  targetCompressionRatio: 0.5, // remove ~50% of tokens per node
  minTokens: 200, // skip short nodes
});

const queryEngine = index.asQueryEngine({
  similarityTopK: 10,
  nodePostprocessors: [compresrPp],
});

const answer = await queryEngine.query({
  query: 'What did the customer say about enterprise pricing?',
});
```

Supports absolute output budgets via `targetToken`. Fail-open by default: if the Compresr API is unavailable, your pipeline continues with the original nodes.

3 Memory block

Plug `CompresrMemoryBlock` into the LlamaIndex.TS `Memory` API to shrink chat history in place — no extra LLM summarization call, no dropped messages.

```
import { Memory } from 'llamaindex';
import { CompresrMemoryBlock }
  from '@compresr/sdk/integrations/llamaindex';

const memory = new Memory({
  tokenLimit: 8_000,
  memoryBlocks: [
    new CompresrMemoryBlock({
      apiKey: process.env.COMPRESR_API_KEY!,
      targetToken: 2_000, // output budget in tokens
      minTokens: 200,
    }),
  ],
});
```

4 Tool wrapper

Wrap any `FunctionTool` so its string return value is compressed before reaching the agent. Metadata (name, description, schema) is preserved; non-string returns pass through unchanged.

```
import { FunctionTool } from 'llamaindex';
import { wrapToolWithCompresr }
  from '@compresr/sdk/integrations/llamaindex';

const search = FunctionTool.from(
  async ({ query }: { query: string }) => fetchCRM(query),
  {
    name: 'search_crm',
    description: 'Search the CRM for customer notes',
    parameters: {
      type: 'object',
      properties: { query: { type: 'string' } },
    },
  },
);
```

```

    required: ['query'],
  },
},
);

const wrapped = wrapToolWithCompressr(search, {
  apiKey: process.env.COMPRESSR_API_KEY!,
  targetCompressionRatio: 0.5,
  queryArg: 'query', // use args.query as the compression query
});

```

Share one `CompressionClient` across components to pool HTTP transport and apply a single timeout / retry config:

```

import { CompressionClient } from '@compressr/sdk';

const client = new CompressionClient({
  apiKey: process.env.COMPRESSR_API_KEY!,
});

const pp = new CompressrNodePostprocessor({ client, targetCompressionRatio: 0.5 });
const memory = new CompressrMemoryBlock({ client, targetToken: 2_000 });
const wrapped = wrapToolWithCompressr(search, { client, queryArg: 'query' });

```

5 Key parameters

Parameter	Default	Description
<code>apiKey</code>	—	Your Compressr API key. Required unless you pass <code>client</code> .
<code>client</code>	<code>undefined</code>	Share a single <code>CompressionClient</code> across the postprocessor, memory block, and tool wrappers for pooled transport, custom timeouts, and proxy / mTLS setups.
<code>baseUrl</code>	<code>undefined</code>	Override the Compressr API endpoint — for self-hosted deployments and Gateway routing.
<code>compressionModel</code>	<code>'latte_v1'</code>	<code>latte_v1</code> (default, query-aware) or <code>latte_v2</code> (faster variant).
<code>targetCompressionRatio</code>	<code>0.5</code>	[0, 1] = fraction of tokens to remove (0.5 = remove ~50%). > 1 = Nx factor (4 = ~4x smaller).
<code>targetToken</code>	<code>undefined</code>	Absolute output token budget per node or buffer. When set, overrides <code>targetCompressionRatio</code> . Effective ratio is computed as $\sim \text{avgNodeTokens} / \text{targetToken}$ and clamped to ≥ 1 , so values larger than the typical node size are a no-op.
<code>minTokens</code>	<code>200</code>	Skip compression when input is below this threshold.
<code>coarse</code>	<code>undefined</code>	<code>true</code> = paragraph-level (faster). <code>false</code> = token-level (finer).
<code>query</code> / <code>queryArg</code> / <code>queryExtractor</code>	<code>undefined</code>	Override or extract the compression query (tool wrapper). Defaults lift <code>query</code> , <code>question</code> , or similar args automatically.
<code>onError</code>	<code>'passthrough'</code>	<code>'passthrough'</code> = log and return original content on failure. <code>'raise'</code> = propagate.